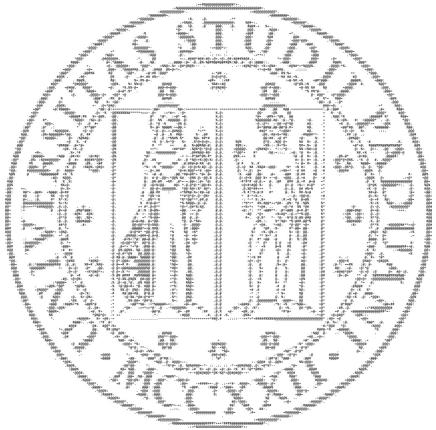
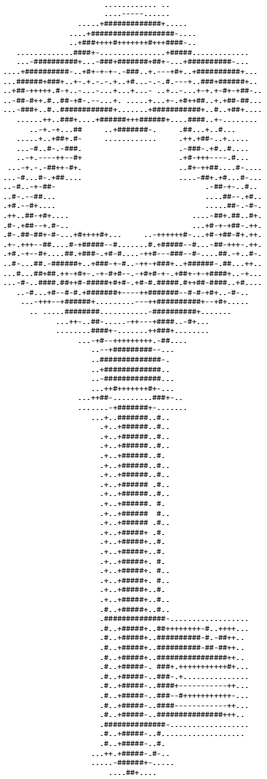
Mitigating Use-After-Free Vulnerabilities in Modern Operating Systems and Browsers: A Comprehensive Survey

COMPUTERS AND NETWORKS SECURITY 2024-2025
Instructor: Mauro Migliardi



Fatemeh Mahvari — University of Padua

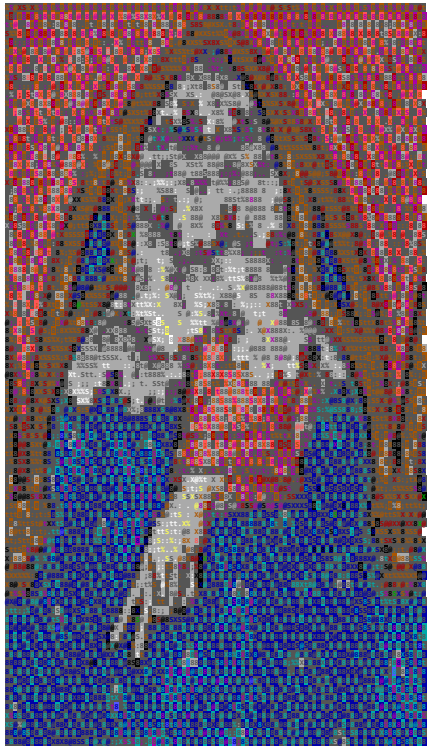
AGENDA



- Introduction & problem scope
- OS-level UAF defenses
- Browser-level mitigations
- Allocator internals (BIBOP, canaries, RIO, type-safe reuse)
- Emerging directions (tagging, capabilities, isolation)
- Case studies
- Trade-offs & effectiveness
- Conclusions & future work

INTRODUCTION

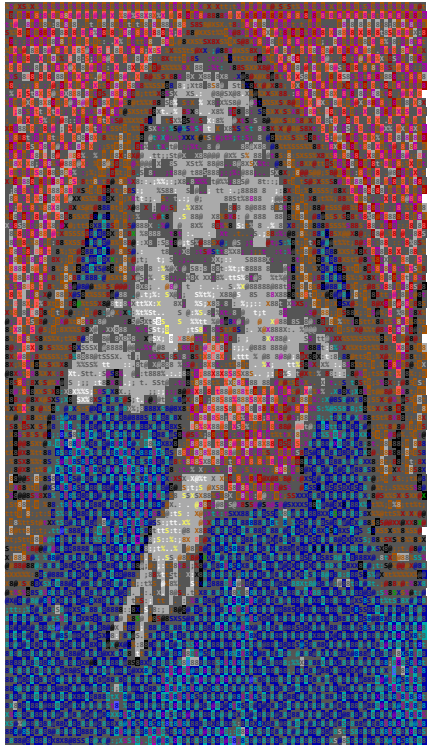
What Is a Use-After-Free (UAF)?



Program frees an object but later uses a stale pointer to it. This **dangling pointer** may now reference reused memory.

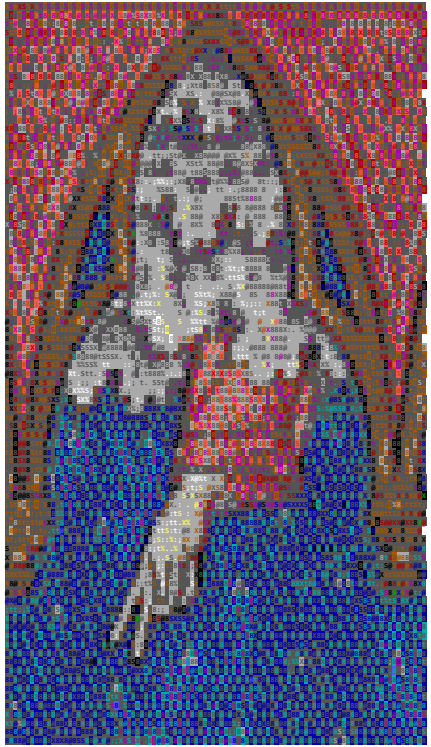
- Leads to arbitrary read/write, RCE, or data leaks in C/C++
- 80% of UAFs rated high severity (2007–2016 study)

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    /* 1. allocate two uint32_t words (8 bytes) */
    uint32_t *p = malloc(2 * sizeof(uint32_t));

    /* 2. create a second pointer that refers to the 2nd word */
    uint32_t *q = p + 1;           // <-- alias to the same block

    /* -- program logic . . . ---------------------------------- */

    /* 3. free the original allocation - p and q become dangling */
    free(p);

    /* 4. attacker (or benign code) forces a new allocation of
          the same size. Very often, the allocator will hand us
          the *same* address.                                    */
    char *u = malloc(2 * sizeof(uint32_t));

    /* 5. stale pointer q still "looks" valid and now corrupts u */
    *q = 0xDEADBEEF;               // <== Use-After-Free write

    printf("u[0..3] now contains 0x%X\n", *(uint32_t *)u);
    free(u);
    return 0;
}
```



Minimal C example of a Use-After-Free vulnerability (adapted from Listing 1 in Bernhard et al.'s xTag paper).

Why UAFs Matter (browsers & kernels)



Large C/C++ codebases → manual memory pitfalls

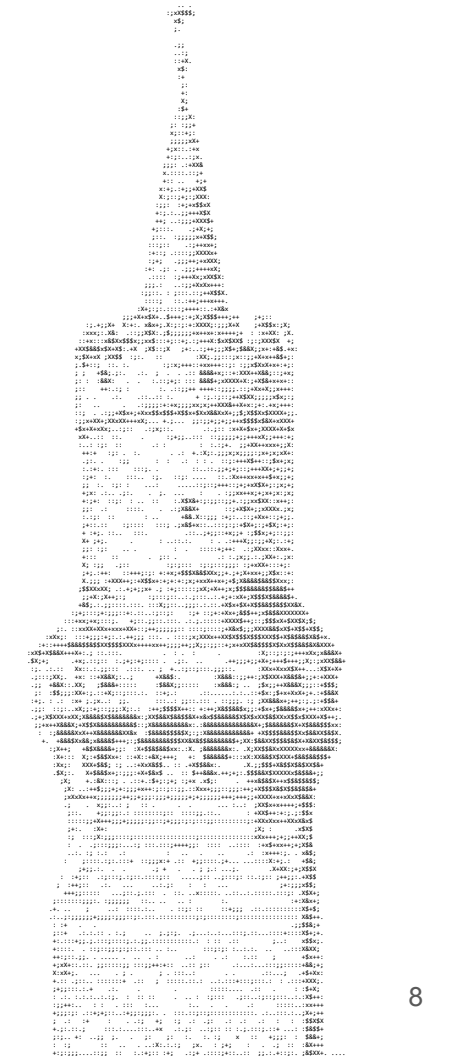
- Chromium: UAF = largest single bug class among high/critical issues
- Firefox: a large share of CVEs historically due to UAF

High privileges + huge attack surface → high impact

OPERATING SYSTEM-LEVEL DEFENSES AGAINST UAF

OS Baseline Defenses: ASLR & W^X

- **ASLR**: randomizes the locations of code, stack, heap, and libraries in a process's address space
- **W^X**: blocks executing code from heap data pages
- Raise attacker cost but don't *prevent* UAF creation
- Attackers pivot to
  - **info leaks**
  - **ROP/JOP**

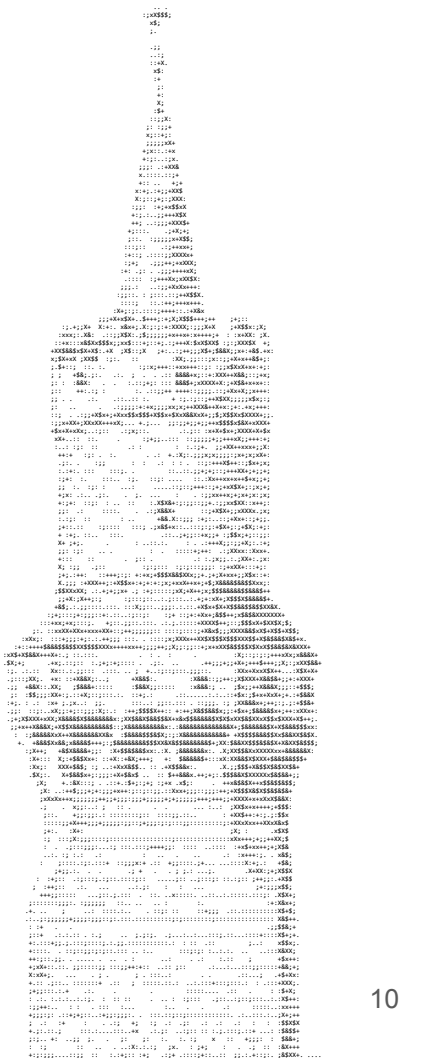


Heap Randomization & Quarantine

- Random free-chunk selection (e.g., Windows LFH)
  - Address obfuscation + random gaps between allocations
- Use of **Quarantine** buffer: delays reuse (e.g., Scudo in Android)
  - ensure the dangling pointer's memory is not immediately recycled for attacker-controlled data
  - More entropy → less deterministic reuse (but still probabilistic)

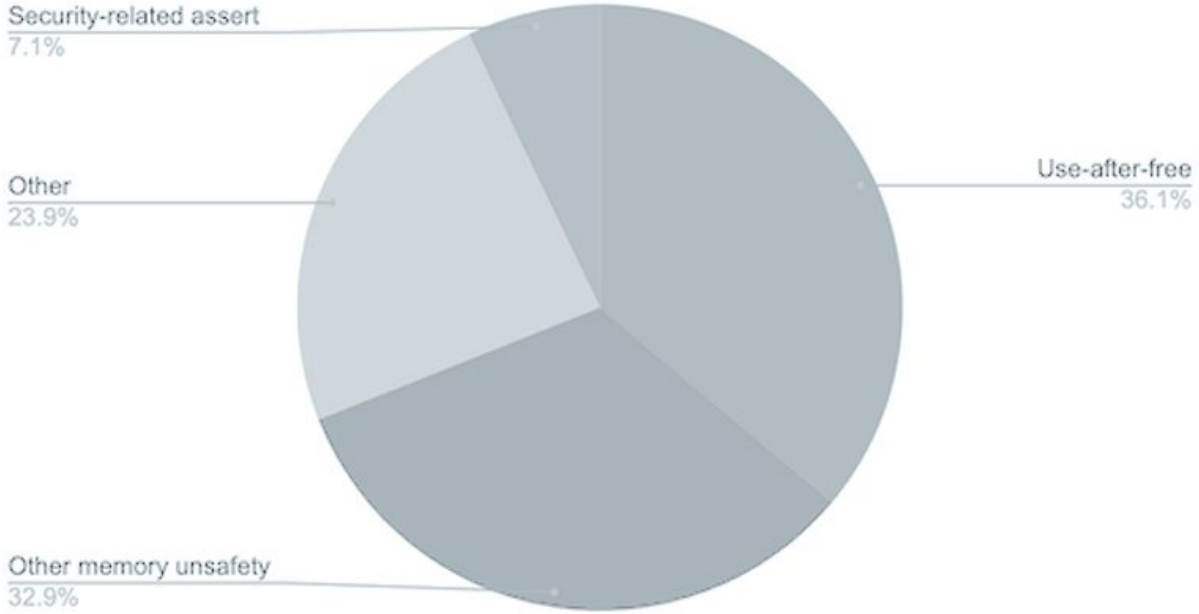
Isolated Heap Regions per Allocation

- **complete partitioning:**
  - isolating objects at page granularity
  - constraining reuse within type-/cache-specific pools
- **Slab architecture** in Linux (What kernels already do)
- Protection keys



BROWSER-LEVEL UAF MITIGATIONS

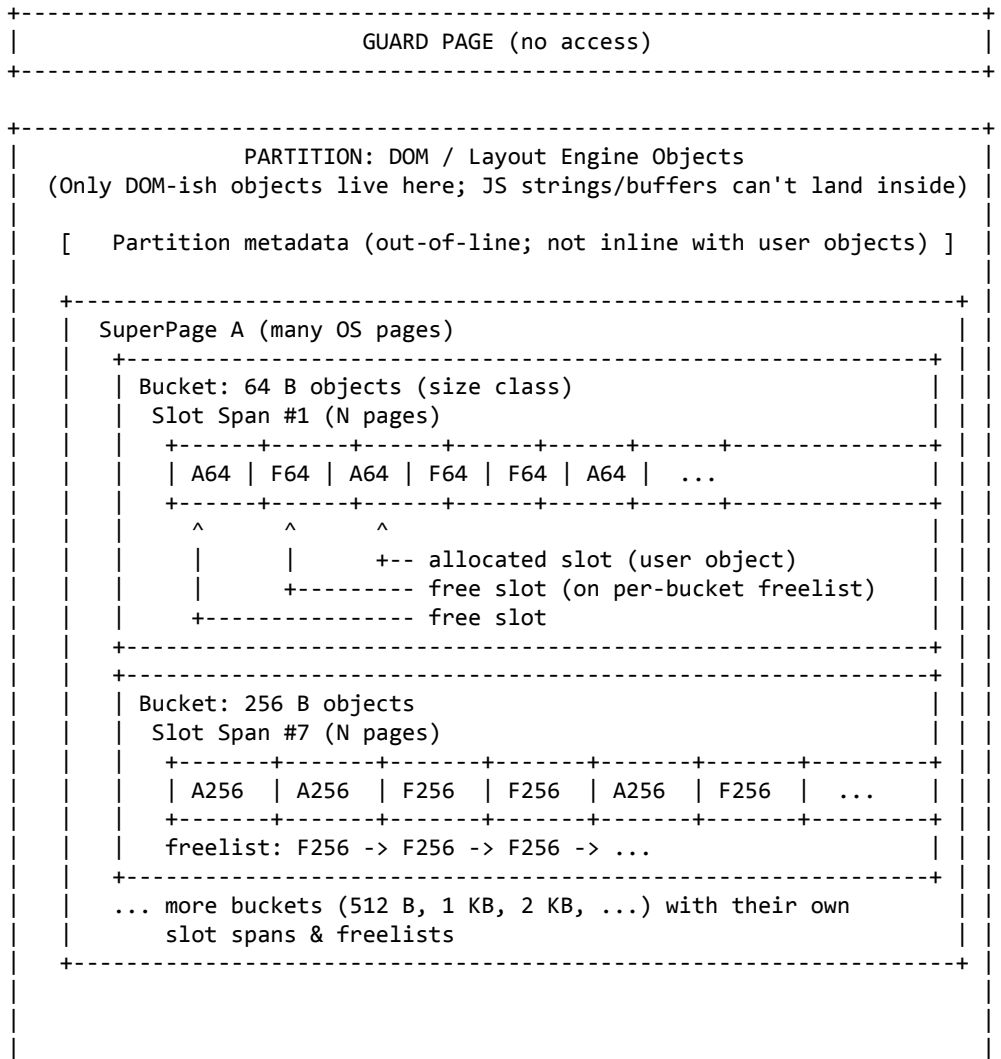
Distribution of High-Severity Chrome Security Bugs



memory unsafety dominates and use-after-free is the largest single category.
Source: The Chromium Projects

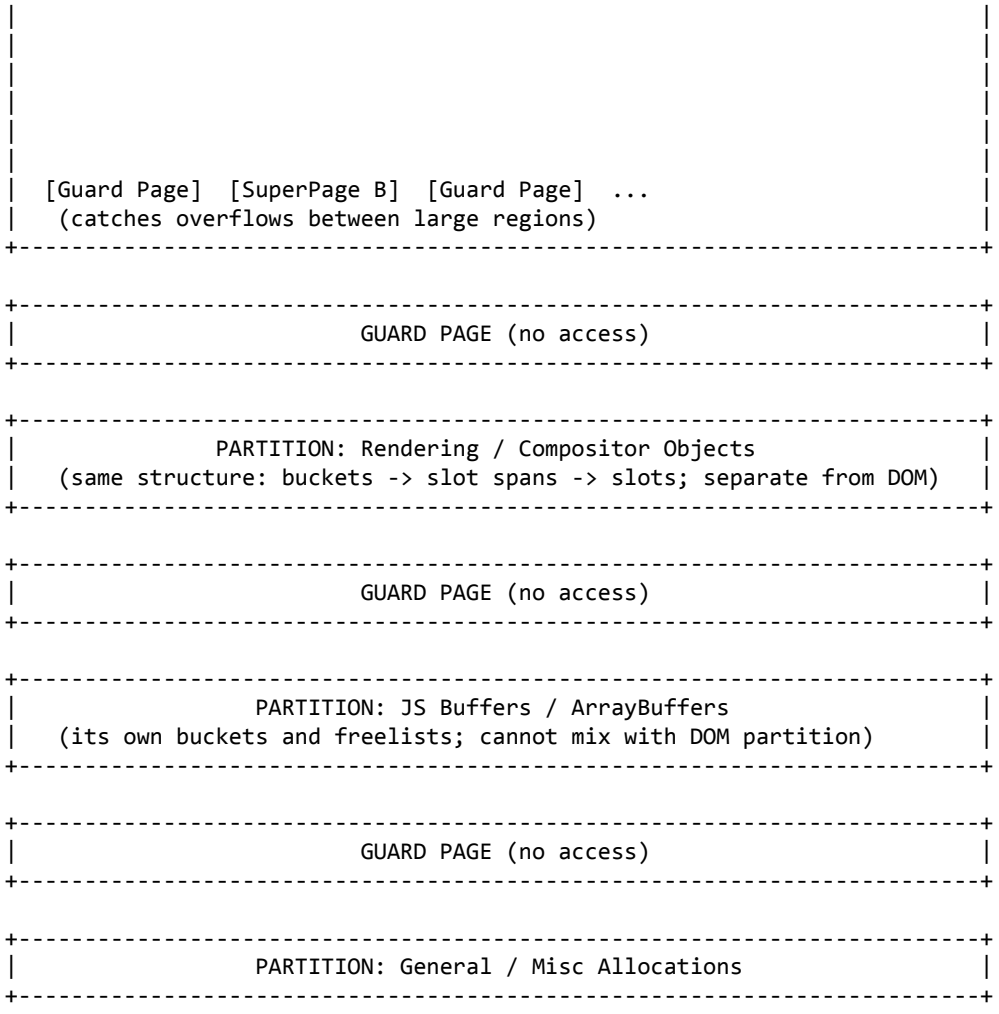
Chrome’s PartitionAlloc

- Partitions heap by **type** (DOM, rendering, JS buffers, misc) + **size classes**
- Prevents cross-type reuse from dangling pointers
- Out-of-line metadata + guard pages harden allocator
- Part of layered defense with site isolation & CFI



Chrome’s PartitionAlloc

Conceptual snapshot of how a PartitionAlloc-style heap can look.
It shows the main moving parts: type partitions, size classes (buckets), slot spans, slots, guard pages, and out-of-line metadata.



Chrome’s PartitionAlloc

Conceptual snapshot of how a PartitionAlloc-style heap can look.
It shows the main moving parts: type partitions, size classes (buckets), slot spans, slots, guard pages, and out-of-line metadata.

IE/Edge Isolated Heap & MemGC

- Most DOM objects are allocated from a dedicated heap
    - if an attacker holds a dangling pointer to a DOM object, they cannot fill that exact heap space with an arbitrary script
- **MemGC** does not immediately release freed objects: **Garbage Collection**

Firefox’s Arena Allocators and Heap Hardening

- **Arenas** recycle fixed-size slots (some type/size segregation)
- **Frame poisoning** fills freed memory with invalid patterns to crash early
- Adoption of hardened allocators (e.g., Scudo on some platforms)
- **RLBox** sandboxing to contain third-party library bugs
- Adopting **Rust** for new components: Project “**Oxidization**”

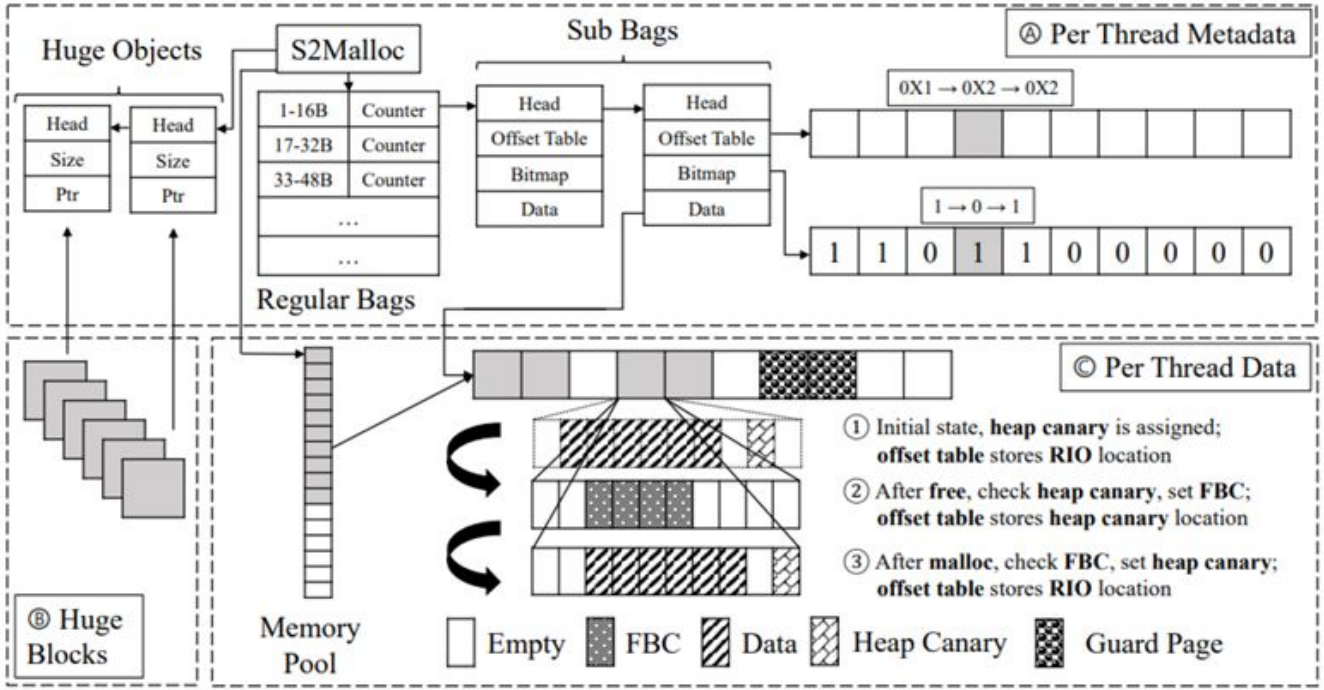
TECHNICAL MITIGATION MECHANISMS IN MEMORY ALLOCATORS

BIBOP Layout

- Separate **bags** per size class; fixed-size **slots** tracked by bitmaps
- Out-of-line metadata prevents metadata corruption
- Natural containment: UAF stays within size-segregated regions
- Guard pages can catch OOB or UAF derefs

Allocators: DieHarder, Guarder, SlimGuard, S2malloc

BIBOP Layout



S2malloc's combo: quarantine + RIO + FBC

Free-Block Canary

- On **free**, write random canary inside the slot
- On **reuse**, verify canary → detect tampering; abort or alert
- Strong against writes; reads-after-free still possible
- Random canary **offset** makes bypass harder

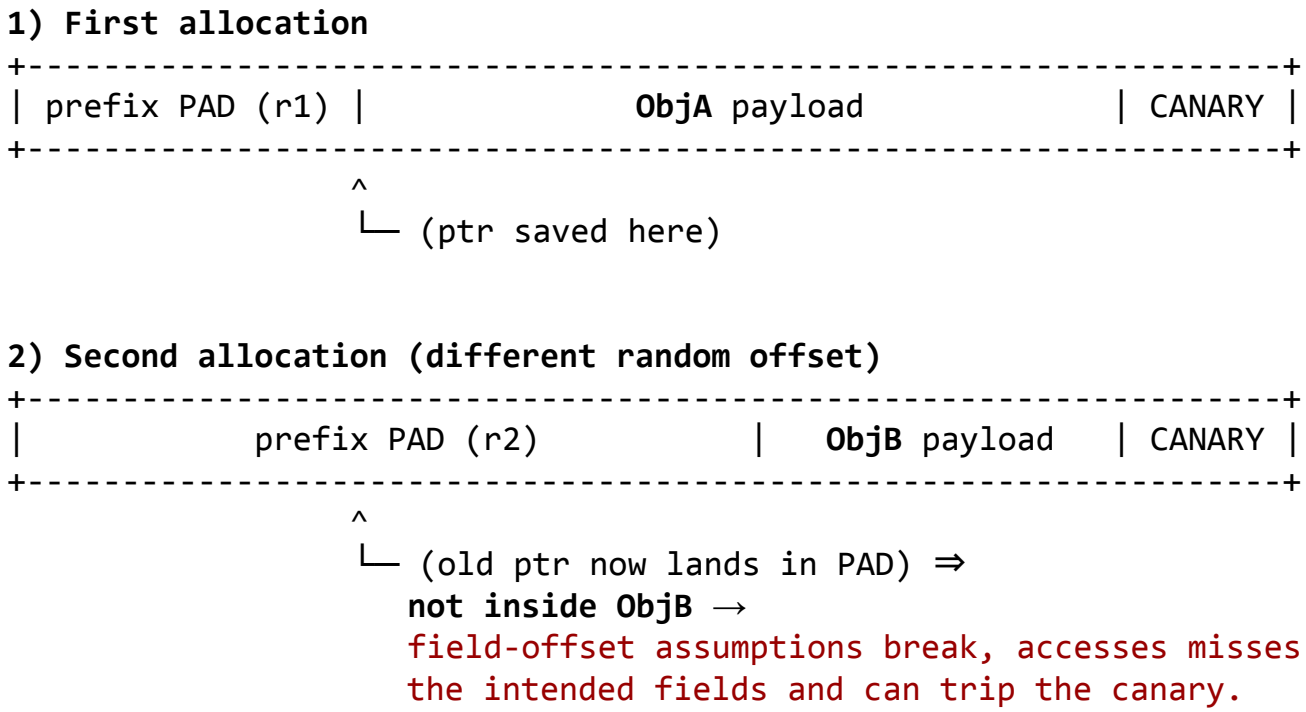
Allocators: S2malloc

Randomized In-Block Offset (RIO)

- Add random prefix padding so object start **shifts** each time
- Breaks attacker’s field-offset assumptions when slot is reused
- Often pairs well with canaries & quarantine
- Security vs memory overhead trade-off

Allocators: S2malloc, ZEUS, Guarder

Randomized In-Block Offset (RIO)



Type-Safe Reuse

- Reuse freed memory **only** for same **type** (or same **call site**)
- Blocks cross-type UAFs (e.g., file object → string corruption)
- Doesn't stop **same-type** UAFs; metadata/complexity overheads
- Designed to integrate with large codebases (e.g., Firefox allocators)

Allocators: Cling, Type-After-Type

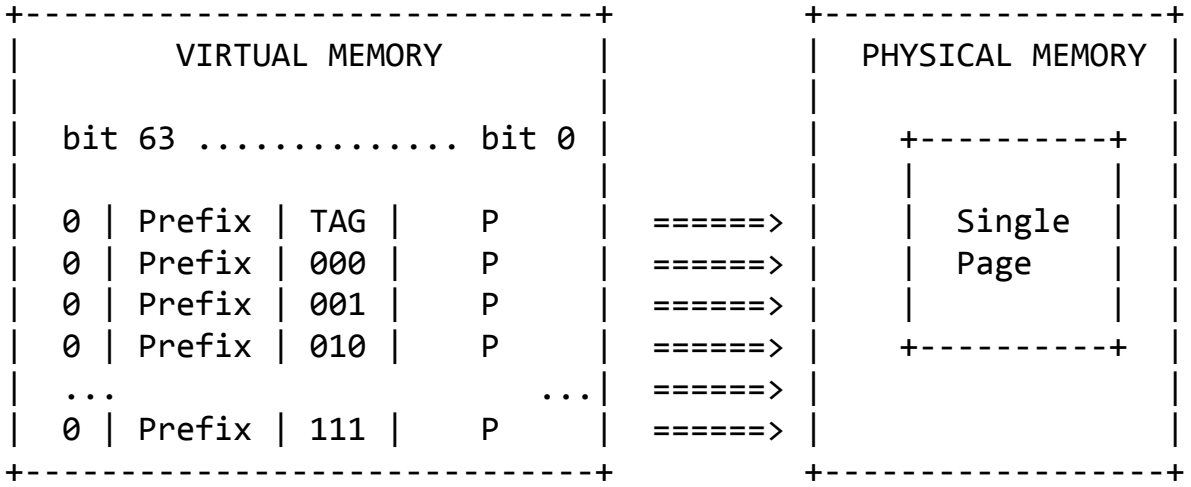
EMERGING DIRECTIONS IN UAF MITIGATIONS

Hardware Memory Tagging (ARM MTE, SPARC ADI)

- Store small **tags** in pointers + memory; mismatch → fault
- Catches UAF by invalidating/retagging on free/reuse
- Low overhead in hardware; limited availability on x86-64
- Browsers/OSes can leverage MTE to raise attacker cost

Software Tagging (xTag and Memory Coloring)

xTag:
uses the processor's MMU to alias multiple virtual pages to the same physical page, embedding tag metadata into pointer-relevant bits without requiring special hardware.



xTag page-aliasing for software pointer tagging: changing the 4-bit metadata (e.g., 0000→0001) alters the virtual address, but all 2^4 aliases map to the same physical page so any tagged pointer still dereferences the same allocation.

Capability Architectures and Pointer Revocation

- **Design goal: address the root cause of UAF by eliminating dangling pointers at the hardware level**
- CHERI design
  - Implementation: CHERInvoke
    - provide fat pointer capabilities that include bounds and permissions
  - Cornucopia
    - architectural revocation system for CHERI heaps aiming for full temporal memory safety by sweeping memory for stale pointers and invalidating them in hardware

Fault Isolation and Compartmentalization

- **Design goal: isolating the effects of UAF errors to the process's sandbox**
  - Chrome's Site Isolation
    - Example: RCE cannot transform into UXSS
  - Out-of-Process JIT compilation
  - Firefox's RLBox

- microkernel or compartmentalization principles to isolate memory regions of different subsystems…

CASE STUDIES OF UAF EXPLOITS AND MITIGATIONS

Case Study #1: mRuby Issue #4001 (CVE-2018-10199)

Problem: `mrb_io_initialize_copy` frees an I/O object and, on certain exceptions, exits without reassigning `DATA_PTR(copy)`, leaving a dangling pointer.

Exploitation: free I/O (A), place string (B) in its slot, then use the stale I/O pointer to overwrite B's critical fields, gaining out-of-bounds read/write via the corrupted string length.

Actual Fix: Upstream patch ("Fix use after free in `File#initilialize_copy`") adjusted initialization/ordering to avoid the dangling pointer

Mitigations: **Type isolation** (e.g., PartitionAlloc/Cling); **free-block canaries (FBC)** and **randomized in-block offsets (RIO)** .

Case Study #2: PHP Session Deserializer UAF (CVE-2015-6835)

Problem: During session decoding, `php_var_unserialize()` returns a new **zval** that is freed by the caller while a reference to the same zval persists in `SESSION_VARS` → dangling pointer.

Exploitation: Heap placement tricks (re-runs/sprays) to force reuse of the freed slot so the stale pointer points into the attacker-controlled zval, enabling memory disclosure or RCE primitives.

Actual Fix: Patched in **PHP 5.4.45 / 5.5.29 / 5.6.13** by correcting the deserializer's handling of multiple `php_var_unserialize` calls (removing the UAF condition).

Mitigations: **FBC**; **RIO**; pure entropy allocators (Guarder/DieHarder) are weaker here; **context/type-based** allocators (**Cling/TypeAfterType**) can fail.

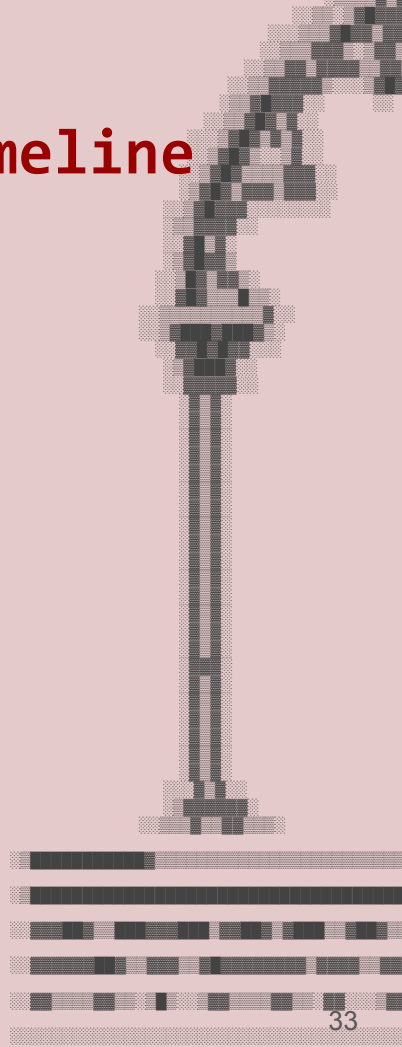
Case Study #3: Firefox Web Animations timeline UAF (CVE-2024-9680)

Problem: During `AnimationTimeline::Tick()`, re-entrancy via JS promise callbacks can cancel/remove an animation while the list is being iterated, leaving a dangling `Animation` pointer.

Exploitation: Exploited in the wild for content-process RCE, then chained with a Windows Task Scheduler EoP (CVE-2024-49039) to escape the sandbox.

Actual Fix: Mozilla's patch holds RefPtr to animations while ticking (stronger lifetime guarantees, preventing them from being freed during iteration), shipped in Firefox 131.0.2 / ESR 128.3.1 / ESR 115.16.1.

Mitigations: **Allocator hardening** (canaries, partitioned heaps); **fault isolation** (e.g., RLBox for 3rd-party libs / process isolation); **memory tagging**.



DISCUSSION: TRADE-OFFS AND EFFECTIVENESS OF UAF DEFENSES

Trade-Offs: Guarantees vs Assumptions

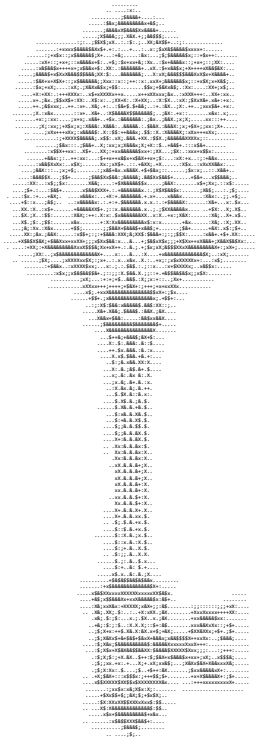
- **Probabilistic** (ASLR, random reuse): cheap, but retry/leaks can win
- **Deterministic** (never-reuse pages, tagging): stronger, higher cost/coverage demands
- Defense-in-depth: combine partitioning + canaries + CFI + sandboxing

Cost and Practicality

- Many modern schemes keep runtime overhead to a few percents
- Memory overhead varies widely (quarantine, isolation)
- Integration complexity matters; production favors drop-in allocators

Mitigation × Properties Matrix

Mitigation	Coverage	Enforcement	CPU	Mem	Deployment	Notes
Guarder (USENIX'18)	Temporal	Probabilistic	~2–3%	—	Research	Entropy-based random slot selection to reduce targeted reuse.
SlimGuard (Middleware'19)	Temporal	Probabilistic (tunable r)	a few %	lower than DieHarder	Research	Space-efficient randomized reuse; tunable entropy.
S2malloc (2024)	Temporal	Hybrid (prob.+detection)	~2.8%	~27%	Research	FBC (write detection) + RIO (prefix misalignment).
Oscar (USENIX'17)	Temporal	Deterministic (never-reuse)	—	~60%	Research	Page-permission tricks to prevent slot reallocation.
FFmalloc (USENIX'21)	Temporal	Deterministic (batched never-reuse)	~2.3%	—	Research	Batching reduces runtime overhead at cost of memory.
PartitionAlloc (Chrome)	Both (spatial + partial temporal)	Deterministic	low	low/neutral	Production	Type/size partitioning, out-of-line metadata, guard pages.
MemGC (IE/Edge)	Temporal	Deterministic (quarantine)	amortized	transient increase	Production	Delayed frees; effectiveness may drop under OOM flush.
FreeGuard (CCS'17)	Temporal	Probabilistic (levels)	—	—	Research	Tunable "security levels"; Guarder-like randomized reuse.



CONCLUSIONS AND FUTURE WORK

Mitigation × Properties Matrix

Mitigation	Coverage	Enforcement	CPU	Mem	Deployment	Notes
DieHarder (CCS'10)	Both (probabilistic)	Probabilistic	a few %	up to ~57%	Research	Classic randomized allocator; influential baseline.
Cling (USENIX'10)	Temporal (type isolation)	Deterministic	~11%	—	Research	Per-type/context pools to block cross-type UAF reuse.
Type-after-Type (ACSAC'18)	Temporal (type-safe reuse)	Deterministic	~4%	—	Research	Reuse only within same type; call-site hash fallback.
Scudo (LLVM/Android)	Temporal	Probabilistic (quarantine)	—	—	Production	Hardened general-purpose allocator (Android 11+ for native code).
`hardened_malloc` (GrapheneOS)	Both (hardened checks)	Deterministic	—	—	Production	Strong checks; accepts higher perf/memory costs.
ZEUS / Protected heap allocation (DSC'18)	Temporal	Probabilistic	—	—	Research	RIO-like per-allocation prefix randomization (browser-focused).
Windows LFH (front-end heap)	Temporal (reuse randomness)	Probabilistic	—	—	Production	Bucketed small allocs; reduces determinism of reuse timing.

Key Takeaways

- **Layering works**
  - Allocator hardening + sandboxing + CFI increasingly turn UAFs into crash-only unless chained.
- **Deploy now (low friction):**
  - Partitioned/isolated heaps, quarantines, free-block canaries; enforce CFI; sandbox/RLBox high-risk libs.
- **Adopt when available:** ARM **MTE**;
  - evaluate **xTag/CHERI**; expand **Rust** for new components; use pointer-invalidation in hotspots.
- Practical allocators usually **≤~5% runtime**; memory costs vary (e.g., S2malloc ~27% RAM; Oscar ~60% RAM).

Deployment Decision Flow

A. **Have hardware tags?** → Enable **MTE** + keep allocator hardening.

B. **No hardware tags?** → Use **quarantine + canary + partitioned/isolated heaps** as baseline.

C. **Threat model includes info-leaks or chains?** → Add **CFI** and stronger **process/compartment isolation**.

Open Research & Next Steps

- Hardware–software co-design for temporal safety (MTE)
- Better automatic pointer invalidation & analyses
- App-specific strategies (browser-aware randomization/tagging)
- Bringing strong checks to kernels without breaking performance

THANK YOU!

Q&A!?